

Mock Paper 1 — Computer Systems (H446/01) — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

General guidance: award marks for any valid equivalent point. Where a question says "three differences", award one mark per distinct correct point up to the maximum. Calculation questions: award method marks even if the final answer is wrong, provided working is shown.

Question 1 (8 marks)

1(a) [3] (AO1) — 1 mark each, max 3: - PC holds the **address of the next instruction** to be fetched. - At the start of fetch this address is **copied from the PC into the MAR**. - The MAR holds the **address of the memory location** to be read (or written); the address bus carries this address to memory. (PC is then incremented.)

1(b) [3] (AO1) — 1 mark each, max 3 (any three): - **Number of cores** — more cores allow more instructions/processes to be executed in parallel. - **Cache size/levels** — a larger cache reduces the need to fetch from slower main memory. - **Word length / bus width** — wider data/address buses move more data per cycle / address more memory. - **Pipelining** — fetching, decoding and executing different instructions simultaneously.

1(c) [2] (AO2) — 1 mark for advantage stated, 1 for explanation in context: - Harvard has **separate memories/buses for instructions and data**, so an instruction and a data item can be fetched **simultaneously** (1), increasing throughput — useful in an embedded/real-time system where speed and predictable timing matter (1). (Accept: instruction and data memories can have different widths/optimisations.)

Question 2 (7 marks)

2(a) [2] (AO1) — 1 each, max 2: no moving parts so more durable/shock-resistant; lower power consumption (longer battery life); faster access/read-write times; silent; lighter.

2(b) [3] (AO2) — 1 mark per valid comparison point, max 3: - Magnetic typically offers **higher capacity per £** than optical. - Optical media (e.g. Blu-ray) are **cheap to mass-produce/distribute**; magnetic (HDD) is re-writable many times more reliably. - Magnetic generally has **faster access/transfer** than optical drives. - Optical is more portable/robust against magnetic fields; magnetic can be erased by strong magnetic fields.

2(c) [2] (AO2) — 1 + 1: - **Solid state / flash memory** (1). - Justification: no moving parts so robust for a portable device, low power, fast write, small and removable (any one valid) (1).

Question 3 (9 marks)

3(a) [4] (AO1) — 1 mark each, max 4: - Memory (physical and virtual) is divided into fixed-size blocks called **pages/frames**. - Pages not currently needed are stored on **secondary storage (the swap/page file)**. - When a program needs a page that is not in RAM (a **page fault**), the OS **swaps** a page out and loads the required page in. - This allows the **logical address space to exceed physical RAM** / programs larger than RAM to run (accept: addresses are translated logical → physical via a page table).

3(b) [3] (AO1) — max 3: - An interrupt is a **signal to the processor** that an event needs attention (from hardware/software) (1). - The current state/registers are **saved (pushed to a stack)** before servicing (1). - The **ISR is the routine that handles that specific interrupt**; afterwards the saved state is restored and execution resumes (1).

3(c) [2] (AO1) — 2 marks for a clear distinction: - **Multi-tasking**: one user appears to run several processes "at once" via time-slicing/scheduling. - **Multi-user**: several users share the resources of one system simultaneously (the OS allocates processor time/resources between them and keeps their data separate).

Question 4 (9 marks)

4(a) [3] (AO1) — 1 per difference, max 3: - Compiler translates the **whole program at once** to produce an executable; interpreter translates and executes **line by line**. - Compiled code **runs faster**; interpreted code is generally slower (re-translated each run). - Compiler reports **all errors after compilation**; interpreter stops at the **first error** encountered. - Compiled executable is **platform-specific** and can be distributed without source; interpreter needs to be present at run time.

4(b) [4] (AO1) — 2 + 2: - **Lexical analysis (2)**: source code is broken into **tokens**; whitespace and comments are removed; identifiers are entered into the **symbol table**. - **Syntax analysis (2)**: tokens are checked against the **grammar/rules of the language** (often building a parse/abstract syntax tree); **syntax errors are reported** if the structure is invalid.

4(c) [2] (AO2) — 1 + 1: - Bytecode is **platform-independent** — the same file runs on any device with the appropriate **virtual machine** (1). - So the developer writes/distributes **once** rather than recompiling for each platform / source code is not exposed (1).

Question 5 (7 marks)

5(a) [4] (AO2) — up to 4 for justified application: - Agile is **iterative**, delivering working increments, so changing requirements can be incorporated each iteration (1) whereas Waterfall fixes requirements up front (1). - **Frequent client involvement/feedback** ensures the app matches evolving needs (1). - Waterfall would require costly return to earlier stages if requirements change late (1); Agile reduces the risk of building the wrong product (1). (Max 4.)

5(b) [3] (AO1) — 1 each, max 3: pair programming; test-driven development / writing tests first; frequent small releases; continuous integration; on-site customer; short iterations/stand-ups; collective code ownership; refactoring.

Question 6 (9 marks)

6(a) [3] (AO2) — inheritance up to 2, encapsulation up to 1 (or 2+1 split, max 3): - **Inheritance**:

`StudentMember` inherits the attributes/methods of `Member` (e.g. `name`, `memberID`) and adds/overrides its own (`course`, `borrowLimit()`) — reuse without rewriting (1–2). - **Encapsulation**: attributes such as `memberID` are kept **private** and accessed only through public methods, protecting the data from invalid external changes (1).

6(b) [4] (AO3) — award: - Private attributes declared (1) - Constructor that initialises both attributes (1) - Public getter/accessor method(s) (1) - `getDetails()` returns/combines the details correctly (1)

Example (accept any valid OOP language):

```
class Member
  private name
  private memberID

  public procedure new(name, memberID)
    self.name = name
    self.memberID = memberID
  endprocedure

  public function getName()
    return self.name
  endfunction

  public function getDetails()
    return self.name + " (" + self.memberID + ")"
  endfunction
endclass
```

6(c) [2] (AO2) — 1 + 1: - Polymorphism: the **same method name behaves differently depending on the object's class** (1). - Example: calling `borrowLimit()` on a `Member` returns the base limit, but on a `StudentMember` the **overridden** version runs and returns a different limit (1).

Question 7 (8 marks)

7(a) [4] (AO3) — trace + result + explanation:

Input `num = 3`, `total = 0`, `one = 1`. The loop adds `num` to `total`, then decrements `num`, repeating while `num ≥ 0` (BRP = branch if accumulator positive, i.e. `≥ 0`).

Pass	num at start	total += num	num after SUB one
1	3	0+3 = 3	2
2	2	3+2 = 5	1
3	1	5+1 = 6	0
4	0	6+0 = 6	-1 → BRP false, exit

- Correct trace of total (1) and of num decrement (1).
- **Output = 6** (1).
- Explanation: it computes the **sum 3+2+1+0 = the sum of all integers from the input down to 0** (i.e. triangular sum) (1).

7(b) [4] (AO1) — 1 + 1 + 1 + 1: - **Immediate**: the operand is the **actual value** to be used (no memory lookup) (1). - **Direct/absolute**: the operand is the **address of the data** in memory; the value at that address is used (1). - Distinction clearly drawn (1). - **Indexed**: the effective address = base address + contents of an **index register**, which makes it easy/efficient to **step through arrays/lists** (1).

Question 8 (10 marks)

8(a) [3] (AO1) — 2 for distinction, 1 for use cases: - **Lossless**: original data can be **perfectly reconstructed**; no data discarded. - **Lossy**: some data is **permanently discarded** to achieve smaller files; original cannot be exactly recovered. - Use cases: lossless for text/program files/ZIP; lossy for streaming audio/video/JPEG images (1 for both).

8(b) [3] (AO2) — 2 + 1: - RLE replaces **runs of repeated identical values** with a single value plus a **count** (e.g. AAAA → 4A) (2). - Ineffective for data with **few/no repeats**, e.g. natural photographic images or already-compressed/random data — may even increase size (1).

8(c) [4] (AO2) — 2 + 2: - **Symmetric**: the **same single key** encrypts and decrypts; key must be shared securely (1). **Asymmetric**: a **public key** encrypts and a different **private key** decrypts (a key pair) (1). - **Digital signature**: the sender creates a **hash/digest** of the message and encrypts it with **their private key** (1); the recipient decrypts it with the sender's **public key** and compares hashes — confirming the sender's identity and that the message is unaltered (1).

Question 9 (10 marks)

9(a) [2] (AO1) — 1 + 1: - **Primary key**: an attribute that **uniquely identifies each record** in a table — e.g. `CustomerID` (in Customer) or `BookingID` (in Booking). - **Foreign key**: an attribute that is a primary key in **another table**, used to link tables — e.g. `CustomerID` in `Booking`.

9(b) [3] (AO3) — SELECT correct fields (1), correct WHERE (1), correct ORDER BY (1):

```
SELECT Surname, City
FROM Customer
WHERE City = 'Leeds'
ORDER BY Surname;
```

(Accept `ORDER BY Surname ASC`.)

9(c) [5] (AO3) — SELECT correct fields (1), both tables in FROM (1), correct join condition (1), correct filter `Days >= 5` (1), valid overall syntax (1):

```
SELECT Customer.Surname, Booking.CarReg, Booking.StartDate
FROM Customer, Booking
WHERE Customer.CustomerID = Booking.CustomerID
AND Booking.Days >= 5;
```

(Accept explicit `INNER JOIN ... ON ...` form.)

Question 10 (10 marks)

10(a) [3] (AO1) — award up to 3 across the four layers (1 per correct role, max 3): - **Application** layer: provides protocols for applications (HTTP, FTP, SMTP) / interface to the user's program. - **Transport** layer: splits data into **segments/packets**, manages end-to-end connection, ports, and reliability (TCP/UDP). - **Internet (Network)** layer: adds **IP addresses** and routes packets across networks. - **Link (Network access)** layer: handles the **physical/MAC** transmission on the local network.

10(b) [4] (AO2) — up to 4: - Browser/resolver first checks a **local cache**; if not found it queries a DNS server (1). - DNS servers are arranged in a **hierarchy**: root servers → **top-level domain (TLD)** servers (e.g. `.com`, `.uk`) → **authoritative** name servers for the domain (1). - The query is passed up/along this hierarchy until the **authoritative server returns the IP address** for the domain (1). - The IP address is returned to the host (and cached) so the browser can connect to the correct server (1).

10(c) [3] (AO2) — 1 + 1 + 1: - **Circuit switching** establishes a **dedicated physical path** for the whole communication (e.g. a phone call) (1). - **Packet switching** splits data into packets that are **routed independently** and reassembled at the destination (1). - Packet switching is used on the internet because it **uses bandwidth efficiently / shares links / reroutes around failures** (no dedicated line tied up) (1).

Question 11 (12 marks)

11(a) [3] (AO2) — method + answer: - +42 in binary = `0010 1010` (1). - Invert: `1101 0101`; add 1 (1). - Result `1101 0110` (1). (Check: $-128+64+16+4+2 = -42$. ✓)

11(b) [5] (AO3) — - Exponent `0011` (two's complement) = **+3** (1). - Mantissa `0.1011000` with point after first bit; bit values from the point: $0.1011000 \rightarrow 1 \times 2^{-1} + 0 + 1 \times 2^{-3} + 1 \times 2^{-4} = 0.5 + 0.125 + 0.0625 = 0.6875$, with a leading `0` sign bit so it is **positive** (1 for mantissa value). - Apply exponent: $0.6875 \times 2^3 = 0.6875 \times 8 = 5.5$ (1 for shifting/multiplying, 1 for final value **5.5**). - **Normalised? Yes** — for a positive number the first two bits are `0` then `1` (`01...`), which is the normalised form; the leading bits differ so no precision is wasted (1).

(Full marks: exponent value, mantissa value, multiplication, final answer 5.5, correct normalised justification.)

11(c) [4] (AO3) —

```
0110 1101    (= 109)
+ 0101 0011    (= 83)
-----
1100 0000    (= 192 unsigned)
```

- Correct bit-by-bit addition / carries (1).

- **8-bit result** 1100 0000 (1).
- Interpreted as signed two's complement: 0110 1101 = +109 and 0101 0011 = +83; sum = +192 which **exceeds +127** (1).
- Therefore **overflow has occurred** — two positives gave a negative result (1100 0000 = -64 in two's complement), i.e. the sign bit changed incorrectly (1).

Question 12 (10 marks)

12(a) [3] (AO1) — 2 for distinction, 1 for uses: - **Stack: LIFO** (last in, first out). **Queue: FIFO** (first in, first out) (2 for both, or 1 each). - Uses (1, any): stack — managing subroutine return addresses / undo / call stack; queue — print/job spooling / keyboard buffer / scheduling.

12(b) [4] (AO2) — up to 4: - In a **linear** queue, once **rear** reaches the end of the array the space freed at the **front by dequeuing cannot be reused**, wasting memory (1). - A **circular** queue treats the array as a ring so freed front slots can be **reused**, making fuller use of fixed memory (1). - The **rear** pointer wraps using **modulo arithmetic**: $\text{rear} = (\text{rear} + 1) \text{ MOD size}$ (1). - So after the last index it returns to index 0 (provided the queue is not full), keeping enqueue $O(1)$ (1).

12(c) [3] (AO1) — 1 each: - (i) Push to stack = **$O(1)$** . - (ii) Linear search = **$O(n)$** . - (iii) Binary search = **$O(\log n)$** .

Question 13 (12 marks)

13(a) [4] (AO3) — truth table (3) + gate name (1):

A	B	NOT A	NOT B	A AND NOT B	NOT A AND B	Q
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	1
1	1	0	0	0	0	0

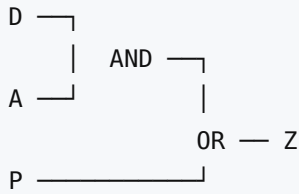
- 3 marks for a correct Q column (deduct 1 per error, min 0; award 1 if at least half correct).
- The expression is equivalent to an **XOR gate** (1).

13(b) [4] (AO3) — award 1 per correct step/law, max 4:

$$\begin{aligned}
 Q &= A \wedge B \vee A \wedge (B \vee C) \vee B \wedge (B \vee C) \\
 &= A \wedge B \vee A \wedge B \vee A \wedge C \vee B \wedge B \vee B \wedge C && \text{(distribution)} \\
 &= A \wedge B \vee A \wedge C \vee B \vee B \wedge C && (A \wedge B \vee A \wedge B = A \wedge B ; B \wedge B = B) \\
 &= A \wedge B \vee A \wedge C \vee B && (B \vee B \wedge C = B, \text{absorption}) \\
 &= A \wedge C \vee B && (B \vee A \wedge B = B, \text{absorption})
 \end{aligned}$$

- **Simplest form:** **$Q = B \vee A \wedge C$** (final answer mark).
- (Accept correct alternative ordering of laws; key laws: distribution, idempotence $B \wedge B = B$, absorption.)

13(c) [4] (AO3) — - Expression: **$Z = (D \text{ AND } A) \text{ OR } P$** (2 marks: AND of D,A = 1; OR with P = 1). - Circuit (2 marks: D and A into an AND gate; output of AND and P into an OR gate; output = Z):



- 1 mark for correct AND gate on D, A; 1 mark for OR gate feeding from the AND output and P to give Z.

Question 14 (7 marks)

14(a) [2] (AO1) — 1 + 1: - **LAN**: covers a **small geographical area** (single site/building) and the **infrastructure is owned by the organisation** (1). - **WAN**: covers a **large geographical area** (between sites/countries) and often uses **third-party/leased** communication links (1).

14(b) [3] (AO2) — up to 3: - A firewall sits between the internal network and external network and **inspects packets** entering/leaving (1). - **Packet filtering** examines each packet's header — e.g. **source/destination IP address and port number** — against a set of rules (1). - Packets that do not meet the rules are **dropped/blocked**, preventing unauthorised traffic/ports from reaching the network (1).

14(c) [2] (AO2) — 1 + 1: - In a **bus** topology all devices share a single backbone, so a break in the cable can bring down the **whole network** (1). - In a **star** topology each device has its own link to a central switch, so if one link fails **only that device** is affected; the rest continue to communicate (1).

Question 15 — Extended response (12 marks) (AO3)

Indicative content (not exhaustive — credit any relevant, well-argued point across legal, ethical, privacy and stakeholder dimensions):

Legal: - Data Protection Act 2018 / GDPR: biometric (facial) data is **special-category personal data** requiring a lawful basis and explicit safeguards; 12-month retention must be justified and proportionate. - Computer Misuse / security obligations to keep the biometric database secure; risk of breach. - Customers' rights: to be informed, to access, and potential lack of meaningful **consent** when scanning is automatic on entry.

Ethical / moral: - **Presumption of innocence** — scanning every customer treats all as potential suspects. - **Algorithmic bias / false positives** — facial recognition is known to be less accurate for some demographic groups, risking wrongful accusation and discrimination. - Transparency and accountability for decisions made by the system.

Privacy: - Mass surveillance of ordinary customers; tracking of movements; potential "function creep" (data reused for marketing/other purposes). - Storing footage and biometrics for 12 months increases the impact of any breach.

Stakeholder perspectives: - **Retailer**: reduced theft/loss, staff safety, cost savings vs reputational and legal risk. - **Customers**: safer stores vs loss of privacy and risk of misidentification. - **Society**: deterrence of crime vs normalising surveillance and eroding civil liberties.

Conclusion: a justified judgement (e.g. deploy only with clear signage, minimal retention, human review of all matches, bias testing and a lawful basis — or do not deploy because consent and proportionality

cannot be met).

Levels of response:

Level 3 (9–12 marks): A thorough discussion covering legal, ethical and privacy issues with clear application to the scenario. Considers **multiple stakeholders** and weighs competing interests. Reaches a **well-justified, balanced conclusion**. Specialist terminology used accurately; response is coherent, structured and sustained.

Level 2 (5–8 marks): Discusses several relevant issues with some application to the scenario. Considers more than one viewpoint but with limited balance/depth. A conclusion is offered but **partially justified**. Some structure and reasonable use of terminology.

Level 1 (1–4 marks): Identifies a few relevant points, largely descriptive/generic with little application to the scenario. One-sided; little or no justified conclusion. Limited structure.

0 marks: No creditable response.

Verified mark total

$8 + 7 + 9 + 9 + 7 + 9 + 8 + 10 + 10 + 10 + 12 + 10 + 12 + 7 + 12 = \mathbf{140 \text{ marks.}}$

Confirmed total: 140 marks.